

HPA Decision Classification Rules

Methodology for the 4-Bucket Correctness Framework

Research Project:	Evaluating Autoscaling Decision Correctness Using Observability Data in Kubernetes
Researcher:	Chamodi Indrejith
Document Version:	1.0
Document Date:	June 2026
Source Script:	analysis/classify_decisions.py
Input Dataset:	results/master_decisions.csv (495 decisions)
Output Dataset:	results/classified_decisions.csv

1. Introduction

This document specifies the classification framework used to evaluate the correctness of Horizontal Pod Autoscaler (HPA) decisions captured during the experimental campaign. Each HPA decision recorded by the custom watcher instrument is assigned to exactly one of four mutually-exclusive correctness buckets based on the rules defined here.

The framework distinguishes between **direction correctness** (did the HPA choose the right direction given the observed metric?), **timing correctness** (did it react with appropriate promptness?), and **action effectiveness** (was the HPA constrained from applying its intended decision?). Together these dimensions form the 4-bucket taxonomy described in Section 2.

2. The Four Correctness Buckets

Bucket	Definition
Correct & Timely	The HPA fired the right scaling action when the metric indicated such action was needed, and did so before the metric significantly overshoot or undershot the configured target.
Correct but Late	The HPA fired the right direction of scaling, but did so only after the metric had moved well beyond the target value. Indicates HPA reaction-time lag for the given workload.
Unnecessary	The HPA fired a scaling action that, based on its own metric snapshot at the time, was not justified (e.g., scaling up when the CPU was actually below target).
Ineffective	The HPA attempted to scale but its intended action was blocked or limited by a hard constraint such as maxReplicas, minReplicas, or another HPA-level cap.

3. Configuration Parameters

The classification rules depend on two configurable thresholds derived from the experimental HPA configuration:

Parameter	Value	Justification
TARGET_PCT	30%	Set to the CPU utilization target configured on the experimental HPA (autoscale-sample-hpa). Decisions are evaluated relative to this threshold.
LATE_PCT	60%	Set to 2x the TARGET_PCT, following common HPA literature convention. A scale-up decision fired only after CPU has reached double the target is considered late-reacting.

4. Classification Rules: Scale-Up Decisions

Scale-up decisions (direction = up) are evaluated by applying the following rules in order. The first matching rule determines the bucket.

Order	Condition	Bucket
1	scaling_limited = True AND reason = TooManyReplicas	Ineffective
2	current_pct < TARGET_PCT (30%)	Unnecessary
3	current_pct > LATE_PCT (60%)	Correct but Late
4	TARGET_PCT ≤ current_pct ≤ LATE_PCT (30% ≤ CPU ≤ 60%)	Correct & Timely

Rationale for the Scale-Up Rules:

Rule 1 (Ineffective) takes precedence because a scale-up that the HPA wanted to perform but could not (due to maxReplicas) is a fundamentally different failure mode than a timing problem. Even if the CPU value would have placed the decision in another bucket, the fact that the action was capped is the primary characterization.

Rule 2 (Unnecessary) catches the unusual case where the HPA scaled up despite CPU being below the configured target. This would indicate either a software defect or a race between metric update and decision. In practice, this bucket is expected to be near-zero on a correctly-functioning HPA.

Rules 3 and 4 (Late vs. Timely) divide the normal scale-up response space at the 2x-target boundary. The choice of 2x reflects the common convention in HPA literature that a CPU value reaching double the target by the time HPA fires represents significant reaction-time lag. Lower thresholds (e.g. 1.5x) were considered but would over-classify normal scale-up behaviors as Late.

Notable Exclusion: ScaleUpLimit:

When the HPA condition shows ScalingLimited = True with reason = ScaleUpLimit, this means the HPA increased the replica count by more than its built-in maximum per-step rate (typically doubling). **This is treated as normal aggressive scale-up behavior, not as Ineffective**, because the HPA did successfully scale - just by more than one step at once. The action took effect; only the magnitude per decision was capped at the per-step limit. The replica count still moved in the correct direction.

5. Classification Rules: Scale-Down Decisions

Scale-down decisions (direction = down) are evaluated by applying the following rules in order:

Order	Condition	Bucket
1	scaling_limited = True AND reason = TooFewReplicas	Ineffective
2	current_pct > TARGET_PCT (30%)	Unnecessary
3	current_pct ≤ TARGET_PCT (30%)	Correct & Timely

Methodological Note on Scale-Down Lateness:

Unlike scale-up classification, scale-down decisions do not have a **Correct but Late** bucket. This is a deliberate methodological choice. The HPA enforces a configurable scale-down stabilization window (default 5 minutes) during which scale-down is intentionally suppressed. As a consequence, every observed scale-down has - by design - waited at least the stabilization window's duration. Distinguishing "designed-slow" from "too-slow" for an individual decision would require a normative reference for the "correct" scale-down latency. No such reference exists in the HPA literature. We therefore retain only the three meaningful buckets for scale-down decisions.

6. Edge Cases and Default Behavior

The classifier handles two edge cases:

Unparseable CPU value: If the trigger_value field cannot be parsed into a numeric percentage (this typically occurs when the trigger metric is memory rather than CPU), the decision defaults to **Correct & Timely**. This conservative default ensures such decisions are not falsely classified as failures. The reason code UP_DEFAULT or DOWN_DEFAULT is recorded to allow filtering in downstream analyses.

Decisions outside any tagged run window: Some decisions fall in idle periods between runs. These are tagged pattern = NULL in the master dataset and excluded from per-pattern statistics, but still appear in the classified output for completeness.

7. Reference Implementation (Pseudo-code)

The following pseudo-code captures the complete classification logic. The production implementation can be found in `analysis/classify_decisions.py`:

```
function classify(decision):
    direction = decision.direction
    current = decision.current_pct
    limited = decision.scaling_limited
    reason = decision.scaling_limit_reason

    TARGET_PCT = 30
    LATE_PCT = 60

    if direction == "up":
        if limited and reason == "TooManyReplicas":
            return "Ineffective"
        if current < TARGET_PCT:
            return "Unnecessary"
        if current > LATE_PCT:
            return "Correct but Late"
        if TARGET_PCT ≤ current ≤ LATE_PCT:
            return "Correct & Timely"
        return "Correct & Timely" # default for unparseable

    elif direction == "down":
        if limited and reason == "TooFewReplicas":
            return "Ineffective"
        if current > TARGET_PCT:
            return "Unnecessary"
        if current ≤ TARGET_PCT:
            return "Correct & Timely"
        return "Correct & Timely" # default for unparseable
```

8. Applied Results Summary

Applying these rules to the 473 HPA decisions tagged to specific workload runs in the experimental campaign produced the following distribution:

Pattern	Correct & Timely	Correct but Late	Unnecessary	Ineffective	Total
Step	94 (75.8%)	27 (21.8%)	0 (0.0%)	3 (2.4%)	124
Burst	89 (69.5%)	19 (14.8%)	0 (0.0%)	20 (15.6%)	128
Ramp	114 (81.4%)	8 (5.7%)	0 (0.0%)	18 (12.9%)	140
Noisy	68 (84.0%)	11 (13.6%)	0 (0.0%)	2 (2.5%)	81
TOTAL	365 (77.2%)	65 (13.7%)	0 (0.0%)	43 (9.1%)	473

Notable observations from the applied classification: (a) all "Unnecessary" counts are zero, indicating the HPA never made a direction-incorrect decision based on its own metric snapshot; (b) the Burst pattern shows the highest Ineffective rate (15.6%), confirming a `maxReplicas` constraint; and (c) Ramp and Noisy patterns exhibit the highest Correct & Timely rates (above 80%), suggesting the

HPA performs best on workloads whose CPU changes are either gradual or, in the case of Noisy, average out over the 60-second metric window.

9. Reproducibility

All data products are reproducible from the source artifacts:

- Source watcher log: results/hpa-events-full.jsonl (522 events)
- Run time-window index: results/run_index.csv (80 runs)
- Tagged decisions: results/master_decisions.csv (495 decisions)
- Classified decisions: results/classified_decisions.csv (495 with bucket)
- Summary counts: results/classification_summary.csv
- Classifier script: analysis/classify_decisions.py
- Tagging script: analysis/build_master_dataset.py

To regenerate the classified dataset from raw inputs:

```
python3 analysis/build_master_dataset.py
python3 analysis/classify_decisions.py
```